UNIT 5 | GENERATIVE DEEP LEARNING

13-mark answers • Architecture, working and clearly labelled diagrams • Based on the uploaded Unit 5 notes

> **Exam clarification:** The notes treat **Text Generation** (pages 2–10) and **DeepDream** (pages 10–15) as separate topics. DeepDream enhances **images**; it is not an architecture for text generation. Because Question 1 mentions both, this answer covers them separately to avoid a wrong exam explanation.

QUESTION 1 | Text Generation and DeepDream – Architecture and Working

PART A. Text Generation – Architecture and Working

1. Introduction

Text generation is a generative deep-learning process in which a model learns language patterns from text data and produces new sentences or passages that resemble human-written language. The model uses earlier words or tokens to predict the next token in a sequence.

2. Architecture of Text Generation

The Unit 5 notes illustrate a Transformer-based language model. Its main components are: input text, tokenization, token and position embeddings, Transformer decoder, Dense output with softmax probabilities, sampling strategy, and feedback of each predicted token into the growing text.

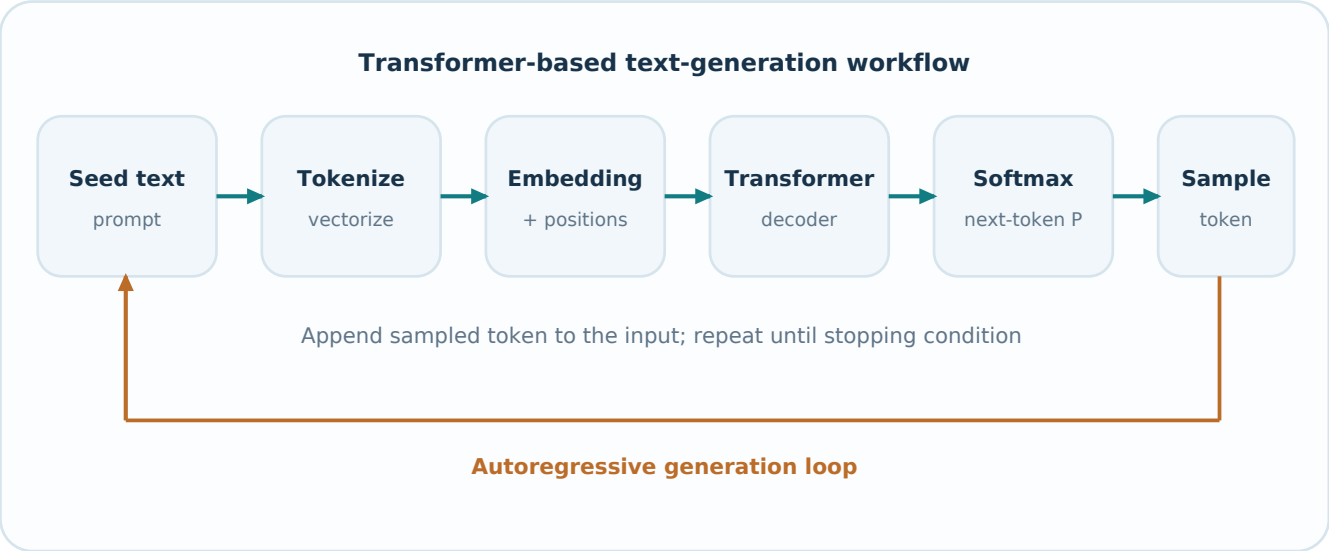


Figure 1. Text-generation architecture and repeated next-token prediction (based on the language-model workflow in the notes, pp. 5–10).

3. Working of Text Generation – Step by Step

- **Step 1 – Prepare the dataset:** Collect text data. The notes use IMDB reviews; text is cleaned and converted into a dataset.
- **Step 2 – Tokenization and vectorization:** Break the text into tokens and convert each token to a numerical ID using a vocabulary.

- **Step 3 – Input/target sequences:** Create input tokens and next-token targets by shifting the sequence by one position.
- **Step 4 – Embeddings:** Convert token IDs into learned vectors and add positional embeddings so the model knows token order.
- **Step 5 – Transformer decoder:** Causal attention learns dependencies while preventing the model from using future tokens during next-token prediction.
- **Step 6 – Probability prediction:** A Dense layer with softmax generates probabilities for the possible next words or tokens.
- **Step 7 – Sampling and repetition:** Select a token, append it to the input, and repeat until the desired length or stopping condition is reached.

4. Sampling Strategies

Strategy	Explanation
Greedy sampling	Always selects the most probable next token; may produce repetitive, predictable text.
Stochastic sampling	Samples randomly according to predicted probabilities, allowing more varied results.
Temperature sampling	Reweights probabilities to control randomness: lower temperatures are more focused; higher temperatures produce more varied samples.

5. Applications

Content creation, chatbots and virtual assistants, language translation, summarization, and coding-related assistance are applications described in the notes.

PART B. DeepDream – Architecture and Working

1. Introduction

DeepDream is an image-visualization and image-enhancement technique that uses a pretrained Convolutional Neural Network (CNN) to identify and amplify patterns in a supplied image. The repeated enhancement of learned features can produce surreal, dream-like images. The notes refer to this as “inceptionism.”

2. Architecture of DeepDream

The model in the notes uses a pretrained **InceptionV3 CNN** with ImageNet weights and without its final classification layers. Intermediate layers, such as **mixed3** and **mixed5**, provide feature activations that are maximized by modifying the input image.

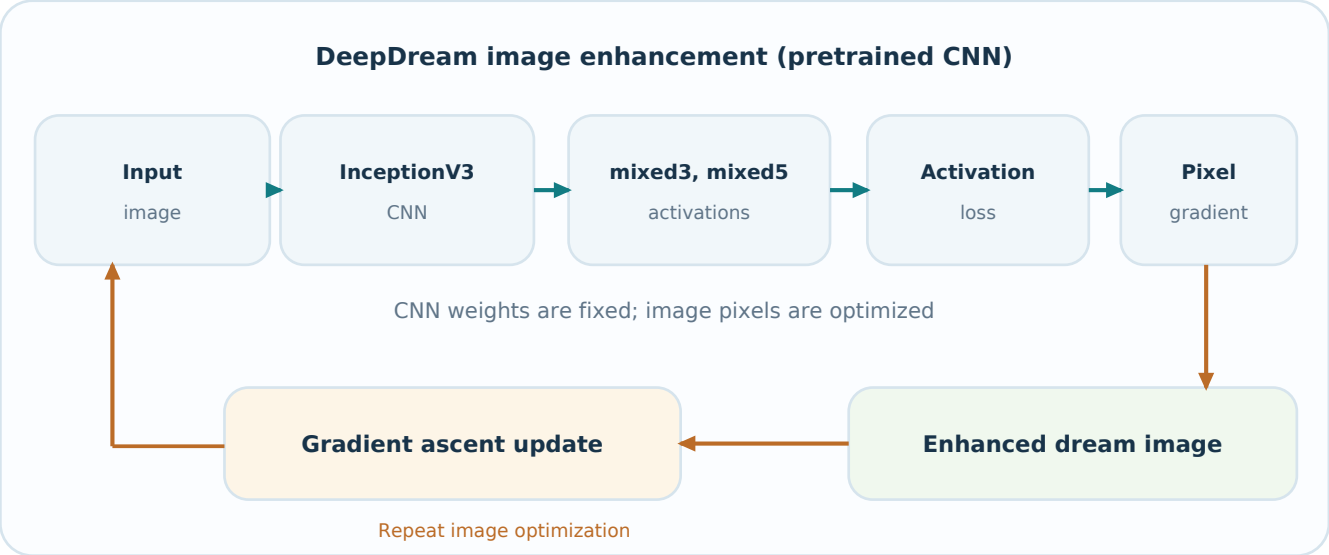


Figure 2. DeepDream architecture: the model's learned features guide repeated optimization of the input image.

3. Main Components

- **Input image:** An image that is resized and preprocessed into the format expected by InceptionV3.
- **Pretrained CNN:** InceptionV3 extracts image features using previously learned weights.
- **Intermediate feature layers:** Selected layers (mixed3 and mixed5) produce activations for different visual patterns.
- **Activation-based objective:** Mean activations from selected layers are combined into a value to maximize.
- **Gradient-ascent image update:** Gradients are calculated with respect to input pixels; pixels are altered to strengthen the detected patterns.

4. Working of DeepDream – Step by Step

- **Step 1 – Load libraries:** Import TensorFlow, NumPy and Matplotlib for model operations, arrays and visualizations.
- **Step 2 – Load pretrained CNN:** Load InceptionV3 with ImageNet weights and remove the classification head (**include_top=False**). Select the mixed3 and mixed5 feature layers.
- **Step 3 – Preprocess input:** Load and resize an image, convert it to an array, and apply the InceptionV3 preprocessing function.
- **Step 4 – Obtain activations:** Send the image through the feature-extraction model to obtain intermediate layer activations.
- **Step 5 – Compute activation loss:** Calculate the mean activation for each selected layer and add the mean values into a single objective.
- **Step 6 – Compute gradients and update pixels:** Calculate gradients of that objective with respect to the image, normalize them, apply gradient ascent, and clip image values to the valid model range.
- **Step 7 – Repeat and display:** Repeat image updates over several iterations to progressively enhance patterns. Convert the processed image back to displayable pixel values to view the final dream image.

5. Important Formula

Objective: $L = \Sigma$ mean(activation of selected layer)

Image update: new image = clip(old image + learning rate × normalized gradient of L, −1, 1)

Key distinction: Standard network training updates the model weights. DeepDream keeps the trained network fixed and optimizes the **input image** by gradient ascent.

6. Result and Conclusion

DeepDream amplifies the visual patterns detected by CNN layers and creates a stylized, dream-like image. Its main purpose in the provided notes is to enhance and visualize learned image features, not to generate sentences.

Source: UNIT 5 DL.pdf, Text Generation pp. 2–10; DeepDream pp. 10–15.

QUESTION 2 | Variational Autoencoder (VAE) for Image Generation

1. Introduction

A **Variational Autoencoder (VAE)** is a probabilistic generative neural network that learns a distribution of the training data. Unlike a traditional autoencoder, which mainly compresses an input and reconstructs it, a VAE learns a structured latent space that can be sampled to generate **new images**.

According to the notes, a VAE contains a probabilistic **encoder**, a sampled **latent vector z**, and a probabilistic **decoder**. The encoder has parameters φ, and the decoder has parameters θ.

2. Architecture of VAE

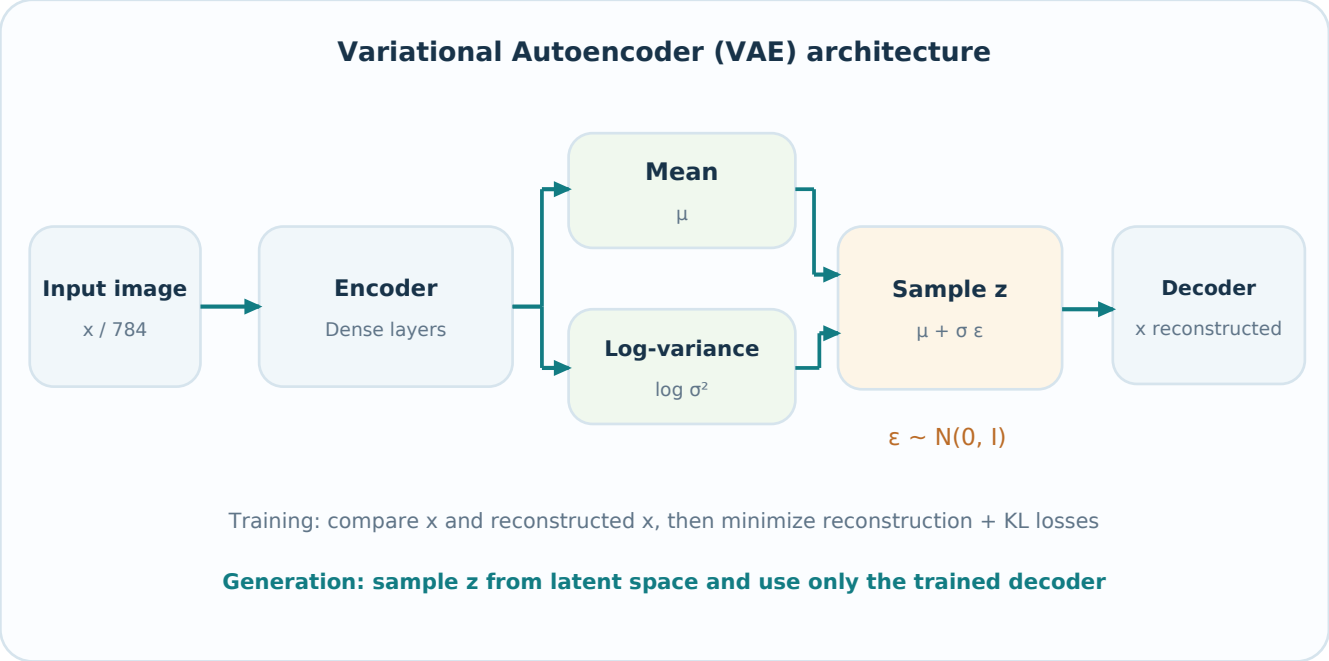


Figure 3. VAE architecture with mean, log-variance, sampling and decoder. Inspired by the VAE diagram on page 25 of the notes.

3. Major Components

- **Input layer:** Receives an image represented as numerical pixel values. In the handwritten-digit example, a 28 × 28 image becomes a 784-value input.
- **Probabilistic encoder:** A neural network extracts features and predicts the parameters of a Gaussian latent distribution, namely mean μ and variance (commonly handled as log-variance).

- **Latent space:** A lower-dimensional continuous space. The notes use a **2-dimensional latent vector** for handwritten-image generation.
- **Sampling/reparameterization:** An auxiliary random variable ε is used to calculate a sampled latent vector while allowing gradients to pass through the encoder during training.
- **Probabilistic decoder:** Converts the sampled latent representation into a reconstructed image, using its learned parameters θ.

4. Reparameterization Trick

Instead of directly sampling z in a way that blocks gradient flow, the VAE uses:

$$z = \mu + \sigma \times \varepsilon \quad \text{where} \quad \varepsilon \sim N(0, I)$$

Here μ is the learned mean, σ is the standard deviation, and ε is random noise from a standard normal distribution. The encoder can output log-variance; σ is obtained from that value.

5. Working of VAE – Step by Step

- **Step 1 – Provide input images:** Training images, such as handwritten digits, are converted to numerical pixel arrays.
- **Step 2 – Pass input through encoder:** The encoder learns image features using Dense layers. The notes show two Dense hidden layers before the latent-parameter outputs.
- **Step 3 – Predict distribution parameters:** The encoder produces **z_mean** and **z_log_var**, which define a probability distribution in latent space.
- **Step 4 – Sample latent vector z:** Apply the reparameterization trick to produce a lower-dimensional latent vector. The example uses a latent dimension of 2.
- **Step 5 – Pass z through decoder:** The decoder uses learned Dense transformations to turn the latent vector into predicted image pixels.
- **Step 6 – Reconstruct the image:** The reconstructed image x′ is compared with original x. The reconstruction should resemble the training image.
- **Step 7 – Calculate training loss:** The VAE uses a loss that combines **reconstruction error** and **KL divergence**. Reconstruction improves image similarity; KL divergence regularizes the latent distribution.
- **Step 8 – Update the network:** Backpropagation updates the encoder and decoder parameters to learn effective reconstruction and a useful latent distribution.
- **Step 9 – Generate new images:** After training, choose or sample new latent points and pass them through **only the decoder**. This produces new images resembling those in the training data.

6. Loss Function of VAE

The notes state that the VAE loss has two terms:

Loss component	Purpose
Reconstruction loss	Measures how closely the generated/reconstructed image matches the original input.
KL divergence loss	Encourages the learned latent distribution to remain close to the assumed prior, typically a standard normal distribution.

Total VAE loss = Reconstruction loss + KL divergence loss

The encoder-decoder system is optimized to balance faithful reconstruction and smooth, useful latent-space sampling.

7. Image Generation After Training

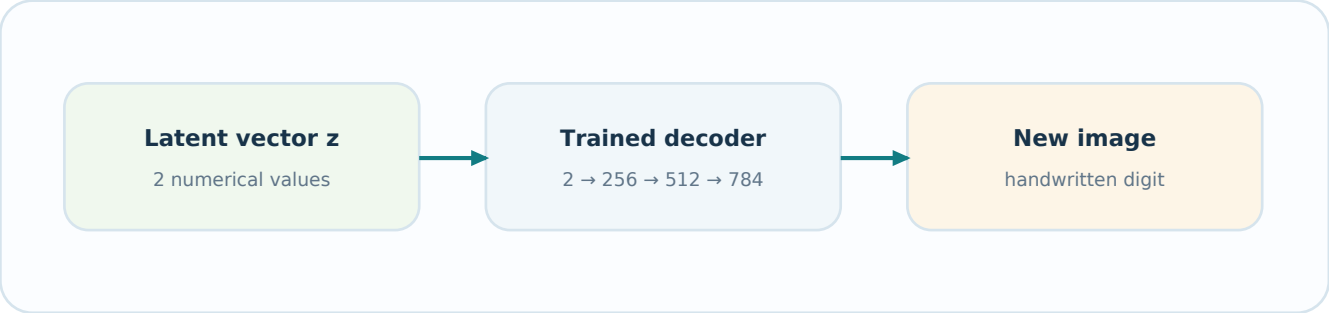


Figure 4. New handwritten-image generation using only the decoder after VAE training.

The notes describe generating handwritten images with a **2-dimensional latent vector**. Different two-number inputs are passed to the trained decoder. The example creates a **196-image interpolation grid** with latent values ranging approximately from −3 to 4.

Example Layer Structure from the Notes

Encoder	Latent space	Decoder
Input: 784 pixels Dense: 512 Dense: 256 Mean + log-variance outputs	2-dimensional sampled vector z	Input: 2 Dense: 256 Dense: 512 Output: 784 pixels

8. Advantages and Applications

- **Generation of new images:** Sampling latent vectors produces images beyond straightforward copies of training inputs.
- **Structured latent space:** Nearby latent values can generate related images, enabling smooth transitions and interpolation.
- **Data representation:** The encoder learns compact probabilistic representations that capture patterns in the training data.

9. Conclusion

VAEs combine an encoder, a probabilistic latent representation and a decoder. By learning a distribution rather than only a compressed code, the trained model can generate new images through sampling. The uploaded notes illustrate this approach by generating many handwritten-digit images from 2-dimensional latent points.

Source: UNIT 5 DL.pdf, VAE introduction pp. 1–2 and Autoencoder/VAE/Image Generation pp. 24–29.